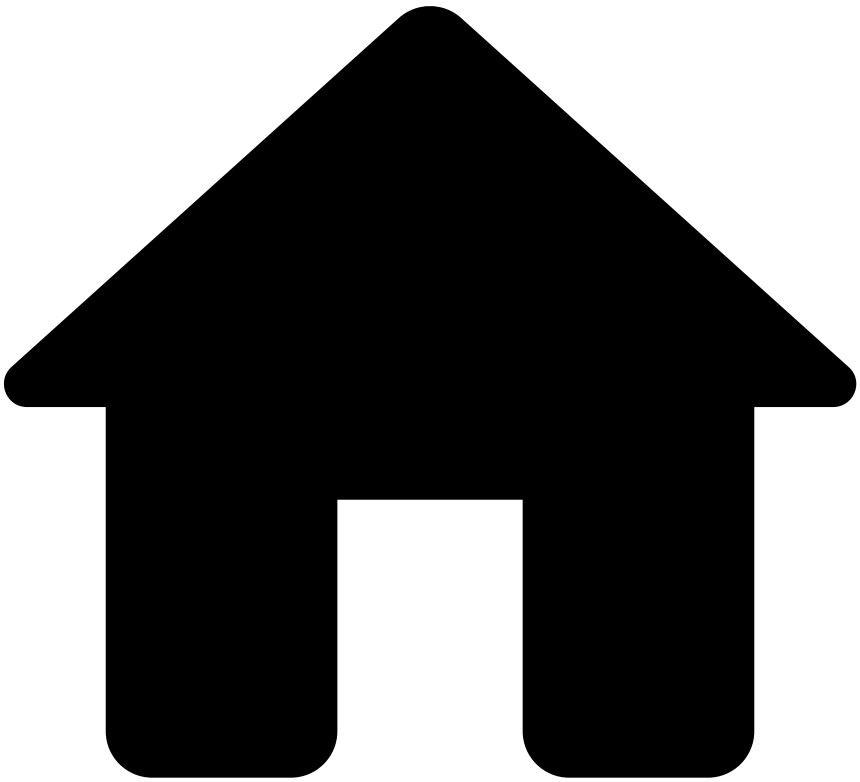


Feedzai FairGBM

Fair Gradient Boosting Machine or FairGBM (FGBM) is a constrained optimization (CO) framework developed by Feedzai on top of Microsoft LightGBM.

FairGBM enables training LightGBM models with fairness constraints (e.g., equality among different ethnicities, genders, age-groups, or any other sensitive attribute). This requires defining a sensitive column that details the protected subgroup each training instance belongs to (note that this column must be in categorical format). FairGBM reduces FPR or FNR

disparities between the different subgroups of the population, according to your choice for the parameter `(Fairness) Constraint type`:

- `FPR` reduces FPR disparities (useful for punitive contexts, such as fraud detection).
- `FNR` reduces FNR disparities (useful for assistive contexts, such as granting loans).
- `FPR,FNR` reduces both FPR and FNR disparities.

In this topic you can learn about the parameters and the usage in Pulse of the Feedzai FairGBM in Pulse.

Feedzai FairGBMâ□□

The biggest advantage of Feedzai FairGBM is that it improves group fairness (e.g., by age, ethnicity, gender), on top of LightGBMâ□□s existing advantages (low memory usage and good performance).

One of its limitations is that training is slightly slower than LightGBM in order to provide fairness.

You can choose from several boosting types:

- **Gradient Boosting Decision Trees (GBDT)**
- **Gradient Based One Side Sampling (GOSS)**: faster than the one above but may yield slightly lower predictive performance.
- **Dropouts meet Multiple Additive Regression Trees (DART)**: can be much slower but might yield higher predictive performance as dropout can yield less over-fitting.

FairGBM in Pulseâ□□

Pulse can train FairGBM models and even import external ones. Only numerical and categorical features are supported. String fields are not used in the FairGBM models.

Numerical features are passed along to the model without transformation, whilst categorical features go through an automatic string-indexing algorithm.

To train a model, FairGBM requires a sensitive column to be selected during train. That column must be categorical. This allows FairGBM to train a fairer model regarding the different population groups available in that column.

Categorical features handling in Pulse for FairGBM modelsâ□□

Since FairGBM can only take in numerical values, categorical features are fully supported by running a simple **string-indexing algorithm at both training and scoring time**, by transforming the category string value into the *nth* index in the lexicographically-sorted list of categories of a categorical variable.

See the example for the string indexing applied to a categorical feature value:

Property	Value
Input value	`"adam"`
Categorical feature categories	`[ "John" , "42", "Adam", "zoe", "adam", "Jane" ]`
Categories in lexicographic order	`[ "42", "Adam", "Jane", "John", "adam", "zoe" ]`
Output value	`4`

Notice that indexes start from 0 and for ASCII character codes, numbers come before letters, and uppercase letters come before lowercase ones.

This arbitrary choice does not affect analytical performance as FairGBM does not consider any order for categorical features.

Training a model in Pulseâ□□

To train a model in Pulse, start a new model training just like any other model type. If you need more details, see the Training a Model page.

All categorical features in the training (and scoring) schema are automatically mapped using the algorithm explained in the section above before being sent to FairGBM. The same mapping is automatically used during scoring.

Numerical features are not transformed.

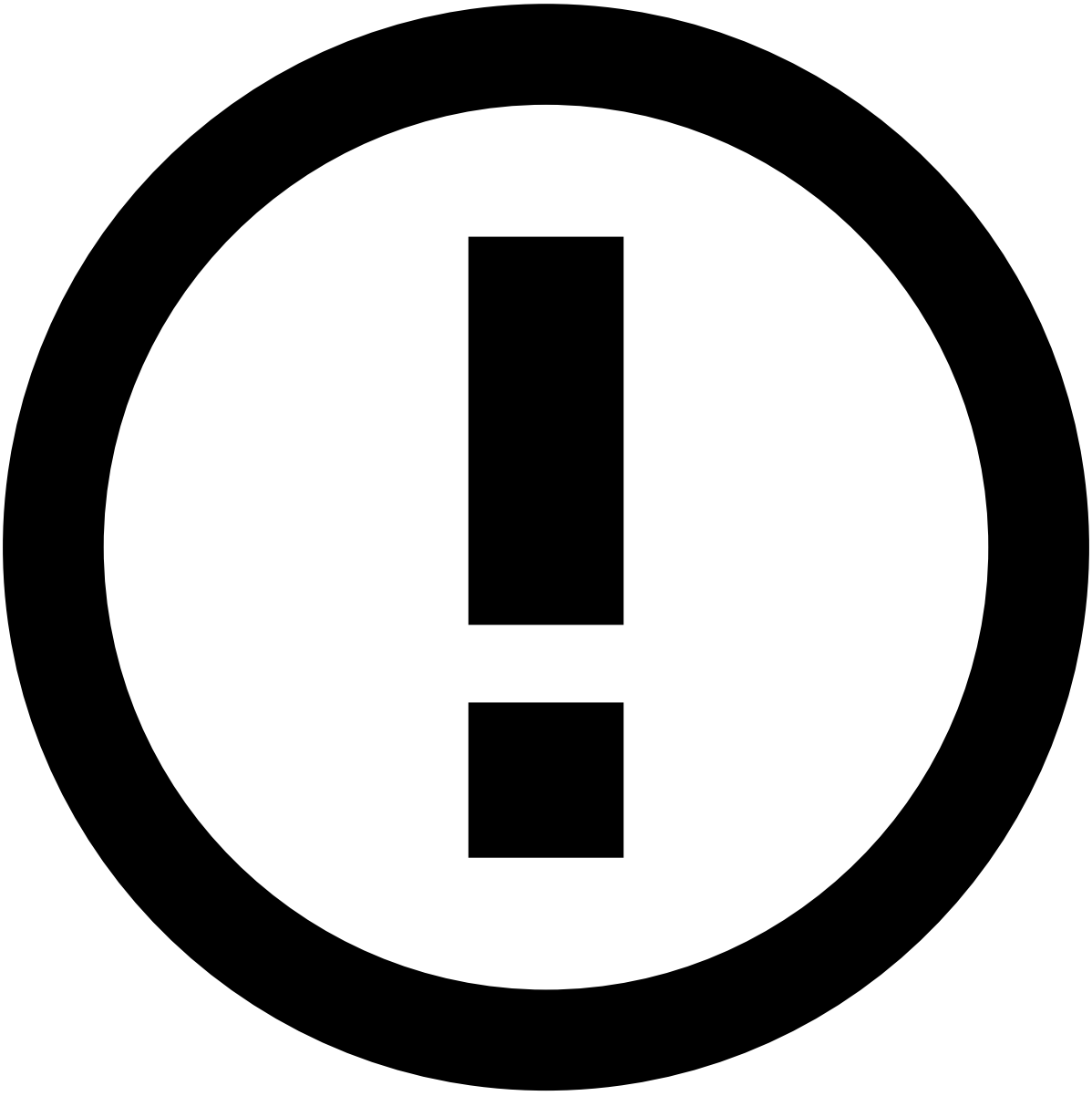
Importing a model to Pulseâ□□

You can import a FairGBM model trained outside Pulse as well.

Model import pathâ□□

To import a model there are two options available:

1. Import the model from a model file path:
    1. The path needs to point to a valid FairGBM model file that FairGBM exports in .txt format.
    2. Since you are not providing the schema, you must define the schema inside the Pulse UI.
2. Import the FairGBM model from a folder path:
    1. The folder pointed to must contain at least the following files:
        1. `FairGBM_model.txt` (mandatory) - the model file as in 1.a.
        2. `model.json` (optional) - a JSON designating the model schema
    2. Since `model.json` is optional:
        1. If not present in the folder, you have to define the schema inside the Pulse UI, as in 1.b.
        2. If present, it will be used for the model schema, unless you override it as in 1.b.



info

Regarding section *2.a.ii*, see this example JSON file for binary classification with both numerical and categorical features.

Importing models with categoricalsâ□□

If an imported model had categoricals in training, you had to index them, and thus that same indexing must now be performed during scoring, in whichever environment is used.

If indexing during training and scoring does not match, the model scores will be affected.

You have several options and must ensure consistency of those transformations:

1. If you declare, in the Pulse model schema, a variable to be categorical, lexicographic order will automatically be used by Pulse for string indexing during scoring:
    - The only way to have correct scores is to train externally in the first place with the same indexing algorithm, otherwise the model's categorical features in training and scoring differ from each other.

2. If you declare, in the Pulse schema, a variable to be numerical (and at the input it is a string that you had to string-index), you're going to have to do the same exact string-indexing transformation for scoring yourself:
    - Make sure you use the same indexing during training and scoring, but that in scoring, such transformations are reliable across environments, with tolerable latencies and memory usage for your use case.

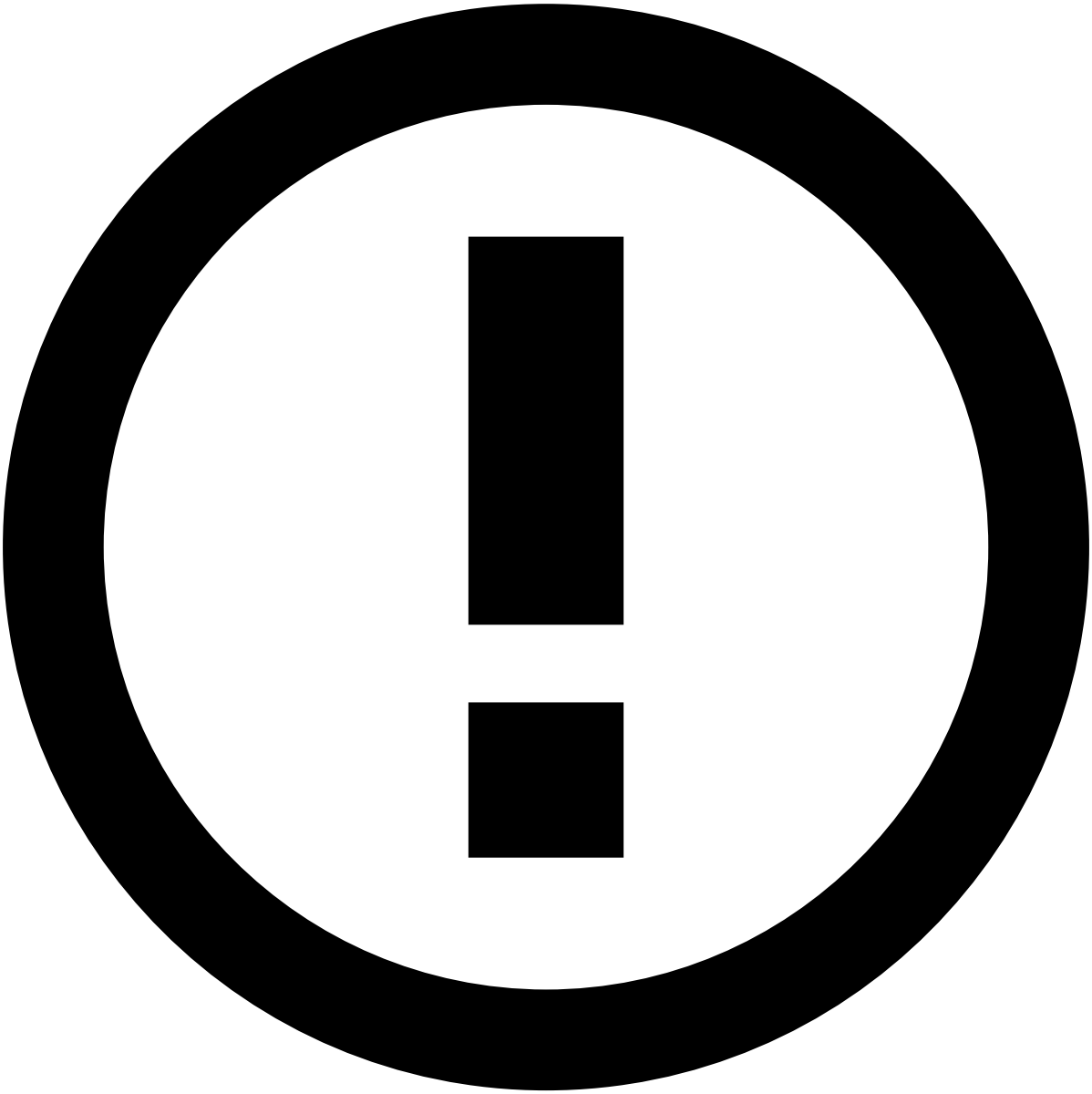
Parametersâ□□

Check the configuration parameters for FairGBM below:

Parameter	Description	Default value
`(Fairness) Sensitive group column`	Fairness constraints are enforced over this column.	(must be provided)
`(Fairness) Constraint type`	Enforces group-wise parity on the given target metric for the selected group column.	`FPR`
`(Fairness) Global constraint type`	Type of global constraint to enforce during training of fairness constraints.	`FPR,FNR`
`(Fairness) Global target FPR`	This is an inequality constraint: inactive when FPR is lower than the target.	`0.05`
`(Fairness) Global target FNR`	This is an inequality constraint: inactive when FNR is lower than the target.	`0.5`
`(Fairness) Objective function`	For FairGBM you must use a constrained optimization function.	`constrained_cross_entropy`
`(Fairness) FPR tolerance for fairness`	The tolerance when fulfilling fairness FPR constraints.	`0.0`
`(Fairness) FNR tolerance for fairness`	The tolerance when fulfilling fairness FNR constraints.	`0.0`
`(Fairness) Multipliers' learning rate`	The Lagrangian multipliers control how strict the constraint enforcement is. **Note that you should use higher values for larger datasets.**	`10000`
`(Fairness) Initial multipliers`	The Lagrangian multipliers control how strict the constraint enforcement is.	`0`
`(Fairness) Stepwise proxy for fairness constraints`	The type of proxy function to use for the fairness constraint.	`cross_entropy`
`(Fairness) Stepwise proxy for global constraints`	The proxy function to use for the objective function.	`cross_entropy`
`Boosting type`	The type of boosting model.	`gbdt`

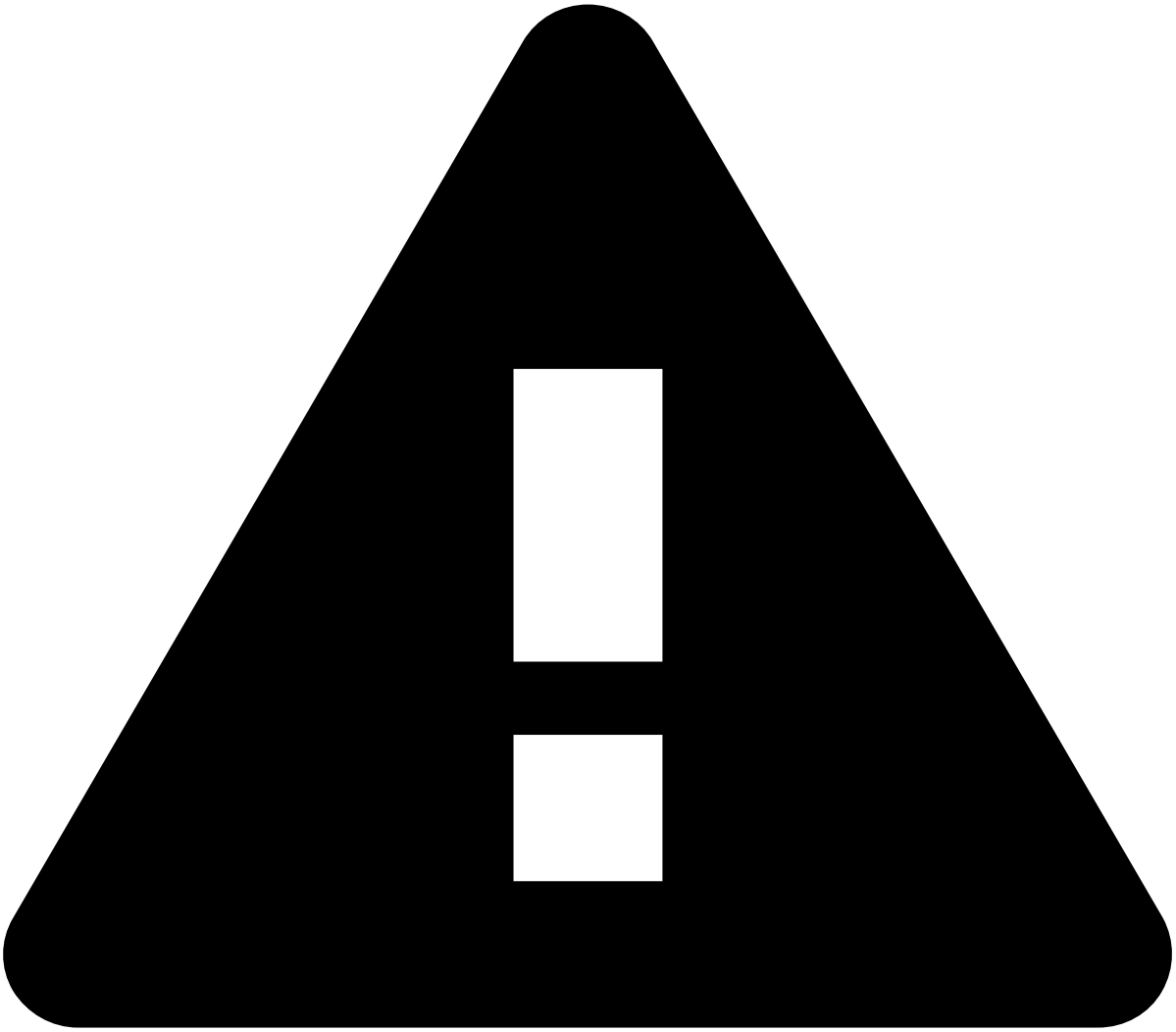
In Pulse, all parameters follow the above specifications, including default values. Handling missing values is always enabled.

Note that there are some parameters that only apply to GOSS and DART boosting modes. Such parameters always have respectively GOSS or DART in their names.



MULTIPLIER TUNING

The Lagrangian multipliers are essential to ensure fairness, and may not be trivial to tune. To tune the multipliers' learning rate, start with the default value (i.e. `10000`), and bear in mind that it roughly increases with the biggest group size. A crude rule of thumb is that the multipliers' learning rate should be between `0.01*n_rows` to `0.1*n_rows`, where `n_rows` is the number of training instances.



CAVEAT

Usually, you choose a decision threshold after training according to one or more of the following metrics: FPR, alert rate, recall, precision, sensitivity, money recall. However, when it comes to FairGBM, this target metric should be provided as a training hyperparameter via `(Fairness) Global target FPR` and/or `(Fairness) Global target FNR`, depending on the optimization goal(s) set in `(Fairness) Global constraint type`. For example, if you want to deploy a fair model at 5% FPR, you must set `"(Fairness) Global target FPR"=0.05` before training. This ensures that your target metric is fulfilled near a threshold of 0.5 (or 500). You can later fine-tune the final threshold after training, but the results should be around 0.5 (or 500).

Logging configurationâ□□

If you want to know in more detail the progress of the process, you can check the LightGBM Logging Configuration page to configure logging to suit your needs.

Learn moreâ□□

- Feedzai Fair GBM code on Github
- Feedzai FairGBM openml provider code on Github
- FAQ on Gradient Boosting Frameworks